

CS462 – Big Data Systems

Lab 2: HDFS & Fault Tolerance

Bellevue College – Spring 2026

Course	CS462 – Big Data Systems
Lab Number	Lab 2
Topics	HDFS, DataNodes, Block Replication, Fault Tolerance
Estimated Time	2–3 hours
Points	130
Submission	Canvas – upload a single ZIP file (see Deliverables section)

Learning Objectives

By the end of this lab you will be able to:

- Spin up a multi-service Hadoop cluster using Docker Compose
- Navigate and manipulate the Hadoop Distributed File System (HDFS) from the command line
- Configure and observe HDFS block replication across multiple DataNodes
- Demonstrate how HDFS tolerates the loss of a DataNode without data loss

Prerequisites

- Docker Desktop installed and running (version 20.10 or later)
- Docker Compose v2 available (docker compose version should print a version number)
- At least 6 GB of free RAM allocated to Docker
- Basic familiarity with the Linux command line

Note: On macOS or Windows, open Docker Desktop → Settings → Resources and raise the memory limit to at least 6 GB before starting the lab.

Environment Setup

Create a new folder called `hadoop-lab` on your computer and place the two files below inside it.

File 1 – `hadoop.env`

This file configures Hadoop by mapping environment variables to the appropriate XML configuration files. The image translates the naming convention SECTION-SITE.XML_property=value directly into Hadoop XML.

hadoop.env

```
CORE-SITE.XML_fs.defaultFS=hdfs://namenode
HDFS-SITE.XML_dfs.namenode.rpc-address=namenode:8020
HDFS-SITE.XML_dfs.replication=1
MAPRED-SITE.XML_mapreduce.framework.name=yarn
MAPRED-
SITE.XML_mapreduce.application.classpath=$HADOOP_HOME/share/hadoop/mapreduce/*:
$HADOOP_HOME/share/hadoop/mapreduce/lib/*
YARN-SITE.XML_yarn.resourcemanager.hostname=resourcemanager
YARN-SITE.XML_yarn.nodemanager.pmem-check-enabled=false
YARN-SITE.XML_yarn.nodemanager.vmem-check-enabled=false
YARN-SITE.XML_yarn.nodemanager.aux-services=mapreduce_shuffle
```

File 2 – docker-compose.yml

docker-compose.yml

```
version: "3"

services:
  namenode:
    image: apache/hadoop:3.4.3
    platform: linux/amd64
    hostname: namenode
    command: ["hdfs", "namenode"]
    ports:
      - 9870:9870 # NameNode Web UI
      - 8020:8020 # HDFS RPC
    env_file:
      - hadoop.env
    environment:
      ENSURE_NAMENODE_DIR: "/tmp/hadoop-root/dfs/name"

  datanode:
    image: apache/hadoop:3.4.3
    platform: linux/amd64
    command: ["hdfs", "datanode"]
    env_file:
      - hadoop.env
    depends_on:
      - namenode

  resourcemanager:
    image: apache/hadoop:3.4.3
    platform: linux/amd64
    hostname: resourcemanager
    command: ["yarn", "resourcemanager"]
```

```
ports:
  - 8088:8088    # YARN ResourceManager Web UI
env_file:
  - hadoop.env
depends_on:
  - namenode

nodemanager:
  image: apache/hadoop:3.4.3
  platform: linux/amd64
  command: ["yarn", "nodemanager"]
  env_file:
    - hadoop.env
  depends_on:
    - resourcemanager
```

Starting the Cluster

Open a terminal, navigate to your hadoop-lab folder, and run:

```
docker compose up -d
```

Wait about 30 seconds, then verify the cluster is healthy by checking the NameNode Web UI at <http://localhost:9870>. Under the Datanodes tab you should see 1 live DataNode.

Note: Apple Silicon Mac (M1/M2/M3): The apache/hadoop image is built for AMD64. If you see a platform mismatch warning or the containers fail to start, add the line "platform: linux/amd64" to each service in docker-compose.yml (directly below the image: line). Docker Desktop uses Rosetta 2 to emulate AMD64 automatically once the platform is specified.

Note: If the NameNode UI shows 0 live DataNodes after 60 seconds, run `docker compose logs datanode` to check for errors.

Part 1: HDFS Basics (45 points)

In this section you will explore the HDFS command-line interface, create a directory tree, upload files, and verify that data can be retrieved correctly.

1.1 Open a Shell on the NameNode

All HDFS commands in this lab are run from inside the NameNode container. Open an interactive shell with:

```
docker exec -it hadoop-lab-namenode-1 bash
```

Note: If your container is named differently, run `docker ps` to find the exact container name.

1.2 Create a Directory Structure

Create the following directory tree in HDFS:

```
hdfs dfs -mkdir -p /user/student/data
hdfs dfs -mkdir -p /user/student/output
```

Verify the directories were created:

```
hdfs dfs -ls /user/student
```

Take a screenshot of the terminal showing the output of the `ls` command. Label it `screenshot-1a.png`.

1.3 Create and Upload a Sample Text File

Inside the NameNode container, create a plain-text file and upload it to HDFS:

```
echo 'To be or not to be that is the question' > /tmp/sample.txt
echo 'Whether tis nobler in the mind to suffer' >> /tmp/sample.txt
echo 'The slings and arrows of outrageous fortune' >> /tmp/sample.txt
echo 'Or to take arms against a sea of troubles' >> /tmp/sample.txt
hdfs dfs -put /tmp/sample.txt /user/student/data/
```

Confirm the file is in HDFS:

```
hdfs dfs -ls /user/student/data
hdfs dfs -cat /user/student/data/sample.txt
```

Take a screenshot showing both commands and their output. Label it `screenshot-1b.png`.

1.4 Download and Verify the File

Retrieve the file from HDFS back to the local filesystem and verify its integrity:

```
hdfs dfs -get /user/student/data/sample.txt /tmp/retrieved.txt  
md5sum /tmp/sample.txt /tmp/retrieved.txt
```

Both MD5 checksums should be identical, confirming the file survived the round-trip through HDFS without corruption.

Take a screenshot showing the md5sum output with matching hashes. Label it screenshot-1c.png.

Part 1 – Deliverables

- screenshot-1a.png – hdfs dfs -ls /user/student output
- screenshot-1b.png – file upload and cat confirmation
- screenshot-1c.png – md5sum showing matching hashes

Part 2: Replication & Fault Tolerance (55 points)

One of HDFS's core guarantees is that data remains available even when individual nodes fail. In this section you will scale up to three DataNodes, observe block replication, and then simulate a node failure.

2.1 Scale to Three DataNodes

Exit the NameNode container (type `exit`), then scale the DataNode service from your terminal on the host:

```
docker compose up -d --scale datanode=3
```

Wait about 20 seconds for the new DataNodes to register, then refresh the NameNode Web UI at <http://localhost:9870>. Click the Datanodes tab and confirm that three live DataNodes are listed.

Take a screenshot of the NameNode UI showing three live DataNodes. Label it `screenshot-2a.png`.

2.2 Increase the Replication Factor

Open a shell on the NameNode again:

```
docker exec -it hadoop-lab-namenode-1 bash
```

Change the replication factor for your sample file to 3 and wait for replication to complete:

```
hdfs dfs -setrep -w 3 /user/student/data/sample.txt
```

The `-w` flag causes the command to wait until replication is complete. Once it finishes, check the file's health:

```
hdfs fsck /user/student/data/sample.txt -files -blocks -locations
```

Look for a line that confirms `replication=3` and `Status: HEALTHY`.

Take a screenshot of the `fsck` output. Label it `screenshot-2b.png`.

Note: `fsck` output can be long. Scroll up to find the block summary lines, or pipe through `grep -E 'replication|Status'`.

2.3 Simulate a DataNode Failure

From a second terminal on your host (not inside the container), first list the running containers to find the exact DataNode container names:

```
docker ps
```

You will see three DataNode containers named after your project folder, e.g. `hadoop-lab-datanode-1`, `hadoop-lab-datanode-2`, `hadoop-lab-datanode-3`. Then stop just one of them (do not use `docker compose stop datanode` — that stops all three):

```
docker stop <project>-datanode-1
```

Refresh the NameNode Web UI. The Datanodes tab should eventually show 2 live nodes and 1 dead node.

Note: This may take up to 10 minutes. Hadoop does not immediately mark a DataNode as dead — it waits for a heartbeat timeout that defaults to roughly 630 seconds (about 10 minutes). Until that threshold is crossed, the NameNode assumes the node is temporarily unreachable. Be patient and keep refreshing; the node will eventually move from the live section to the dead section.

Despite the lost node, your file should still be fully accessible because it was replicated to all three nodes. Verify this from inside the NameNode container:

```
hdfs dfs -cat /user/student/data/sample.txt
```

The file contents should eventually print successfully, demonstrating HDFS fault tolerance.

Note: You may see a `java.net.NoRouteToHostException` error before the file contents appear. This is expected: if the NameNode hasn't yet detected the stopped DataNode as dead, it may direct the client there first. The client will automatically retry using one of the remaining live DataNodes and succeed.

Take a screenshot showing: (a) the NameNode UI with 1 dead DataNode, and (b) the successful cat output. Label it `screenshot-2c.png`.

Part 2 – Deliverables

- `screenshot-2a.png` – NameNode UI showing 3 live DataNodes
- `screenshot-2b.png` – `fsck` output confirming replication factor of 3
- `screenshot-2c.png` – NameNode UI with 1 dead node + successful file read

Tearing Down the Cluster

When you are finished, shut down all containers and free the resources:

```
docker compose down
```

Note: This removes the containers but does not delete your docker-compose.yml or hadoop.env files. You can bring the cluster back up at any time by running docker compose up -d again.

Submission Instructions

Create a ZIP file named lab2_<YourLastName>_<YourFirstName>.zip containing:

1. A write-up document (PDF or DOCX) that includes:
 - Your name, student ID, and the date
 - A brief description (2–3 sentences) of what you did in each Part
 - Answers to the reflection questions below
2. All screenshots labeled as described in each Part (screenshot-1a through screenshot-2c)
3. Your docker-compose.yml and hadoop.env files

Reflection Questions

Answer these questions in your write-up (2–4 sentences each):

4. What is the role of the NameNode vs. a DataNode in HDFS? What would happen to the cluster if the NameNode failed?
5. Why did increasing the replication factor require three DataNodes to be running? What would happen if you tried to set replication=3 with only one DataNode?
6. Based on your fault tolerance experiment, how does HDFS decide which replica to serve when a client reads a file?

Grading Rubric

Task / Deliverable	Points	Grading Criteria
Part 1 – HDFS Setup	10	Docker Compose cluster starts with NameNode and at least one DataNode registered
Part 1 – Directory creation & file upload	20	Screenshot of <code>hdfs dfs -ls</code> confirming directory structure and uploaded files
Part 1 – File retrieval & verification	15	Downloaded file matches original (md5sum comparison shown)
Part 2 – Scaled to 3 DataNodes	15	NameNode UI screenshot shows 3 live DataNodes
Part 2 – Block replication observed	20	<code>hdfs fsck</code> output shows replication factor of 3 for target file
Part 2 – Fault tolerance demonstration	20	One DataNode stopped; NameNode UI shows 2 live nodes; file remains readable
Reflection Questions	30	3 questions × 10 pts each; answers must be 2–4 sentences and demonstrate understanding of HDFS concepts
TOTAL	130	

Note: The due date for this lab is posted on Canvas. Submissions received after that date will receive reduced points in accordance with the course's late submission policy.